

AI Security 1.3. AI Models & Data

Task 1 - Introduction

Every AI model is, at its core, a product of its training data. Before a single prediction is made or a single prompt is answered, decisions about what data to collect, where to collect it from, and how to process it have already shaped everything the model will ever do. Those decisions carry security implications that most organisations deploying AI today have never examined. From PII scraped from the open web to credentials baked into model weights to safety mechanisms quietly eroded during fine-tuning, the risks don't begin when a model is deployed.

PII Personally Identifiable Information is any representation of data that can be used to identify an individual directly.

Learning Objectives

- Understand where AI training data comes from and the security risks introduced by poor data provenance
- Recognise how PII and sensitive credentials can become permanently embedded in model weights through large-scale web scraping
- Understand how key model-building decisions (overfitting, quantisation, and federated learning) each introduce distinct security risks
- Understand the inheritance problem and what organisations unknowingly take on when fine-tuning pre-trained models
- Recognise why trained models are opaque black boxes, and what model cards do (and fail to do) to address this

Task 2 - Training Data

Where Does the Data Come From?

Source	What it is	Trust profile
Web scraping	Automated crawls of public internet content (news, forums, blogs, social media, etc.)	Low: no curator, no version control, content changes after collection

Source	What it is	Trust profile
Licensed datasets	Data purchased or agreed with platforms (e.g., OpenAI + Reddit, Meta's own social posts)	Medium: terms often unclear; original users rarely consented to training use
Synthetic data	AI-generated content used to train further AI systems	Variable: growing fast; ~12% of fine-tuning datasets now contain LLM-generated content
Internal corpora	Company knowledge bases, support transcripts, clinical notes used for fine-tuning	Higher: organisation has direct control, but also direct liability if mishandled

The most widely used training dataset is **Common Crawl** ([opens in new tab](#)), a free, publicly available archive of web crawl data that has underpinned essentially every major model family. DeepSeek-V2 was pretrained on it; DeepSeek-V3 trained on 14.8 trillion tokens with Common Crawl as a core source; and LLaMA 4 was scaled to 40 trillion tokens across 200 languages using a similar pipeline. GPT-3 is one of the few models whose breakdown is publicly documented: 60% of its tokens came from a filtered version of Common Crawl, and more recent models lean even more heavily on it.

Data Provenance: describes the ability to answer where data came from, when it was collected, and whether it has been modified.

The Problem of Provenance

Data provenance is the ability to answer three questions about any piece of training data:

1. Where did it come from?
2. When was it collected?
3. Has it been modified since?

The AI equivalent is the **ML-BOM**: a documented inventory of dataset sources, licenses, PII categories, and filtering decisions. Adoption is still early, and most organisations deploying third-party models today have nothing close to one.

PII in the Pipeline

One of the most direct consequences of undocumented, large-scale web scraping is that personally identifiable information ends up baked into model weights. Once

it's there, it's very difficult to remove.

Answer the questions below

What term describes the ability to answer where data came from, when it was collected, and whether it has been modified?

Data Provenance

What is the name of the most widely used public corpus that underpins essentially every major model family?

Common Crawl

What is the AI equivalent of a Software Bill of Materials (SBOM), used to document dataset sources, licenses, and filtering decisions?

ML-BOM

Task 3 - Training Data

Epochs and Overfitting

An **epoch** is one complete pass of the training algorithm through the entire dataset. In practice, models are trained over many epochs. The algorithm repeatedly sees the same data, adjusting its parameters each time until it converges on accurate predictions.

Overfitting is one mechanism by which a model can "memorise" specific details from its training data, including sensitive ones, making it more likely to reproduce them when prompted.

Model Validation

To catch overfitting early, a portion of the training data is held back and never used for training; this is the **validation set**. At regular intervals during training, the model is tested on unseen data to check whether its performance is actually generalising or just improving on the training examples

Post-Training Optimisation: Pruning and Quantisation

Technique	What it does	Security consideration
Pruning	Removes parameters that contribute little to predictions, shrinking model size	Changes model behaviour post-training; rarely documented in detail
Quantisation	Reduces numerical precision of weights (e.g., 32-bit to 8-bit floats) to cut memory and compute requirements	Can degrade safety-aligned behaviour; backdoor defences tested on full-precision models may fail to detect threats in quantised versions

Federated learning flips this: the model is trained across many decentralised devices or organisations, with each participant training locally on their own data and only sending weight updates (not the raw data itself) back to a central server for aggregation.

Answer the questions below

What term describes one complete pass of the training algorithm through the entire dataset?

Epoch

What problem occurs when a model memorises training data rather than learning general patterns?

Overfitting

What post-training optimisation technique reduces the numerical precision of model weights to cut memory and compute requirements?

Quantisation

What training approach trains a model across decentralised devices, sending only weight updates rather than raw data to a central server?

Federated Learning

Task 4 - The Inheritance Problem

Pre-Trained Models & Fine-Tuning

A pre-trained model is one that has already been trained on a large, general-purpose dataset (the kind of web-scale corpus discussed in Task 2). These base models learn broad language understanding: grammar, facts, reasoning patterns, and world knowledge. They're produced by a small number of well-resourced organisations and then made available for others to build on, either through open weights (like Meta's LLaMA family) or through API access (like OpenAI's GPT series).

Fine-tuning is the process of continuing to train one of these pre-trained models on a smaller, task-specific dataset. A healthcare company might fine-tune a base model trained on clinical documentation to improve its understanding of medical terminology. A law firm might fine-tune on case law. The result is a model with the broad capabilities of the base model, now specialised for a particular domain or use case.

The Inheritance Problem

This is where the security concern sits. When you fine-tune a pre-trained model, you inherit everything that model already contains. This includes things you cannot see and did not choose:

- Biases baked in during pre-training persist
- Unexpected behaviours introduced by the base model's training data carry through
- Any safety alignment built into the base model is not as durable as it might appear

Inheritance Tax

When your organisation deploys a fine-tuned model, you're not deploying the fine-tuning work your team did; you're deploying the entire pre-trained base beneath it. That base was shaped by a training process you didn't control, on data you didn't audit, by an organisation whose supply chain you almost certainly haven't reviewed. Fine-tuning is powerful, but it doesn't sanitise what came before it.

Answer the questions below

What is the process of taking a pre-trained model and continuing to train it on a smaller, task-specific dataset?

Fine-tuning

What term describes a model that has already been trained on a large general-purpose dataset?

Pre-trained Model

Task 5 - The Black Box Problem

Model Cards: documentation artifact accompanies a model to describe what it is, how it was built, and where it falls short.

Weights: the billions of floating-point numbers that make up a trained model collectively referred to.

A well-formed model card should give you the answers to the questions you can't get by inspecting the weights themselves:

Section	What it should tell you
Training data	What sources were used, how they were filtered, known gaps or biases
Intended use	What the model was designed for (and explicitly what it wasn't)
Evaluation results	Performance metrics across different conditions and demographics
Known limitations	Conditions under which the model is known to underperform or behave unexpectedly
Bias assessment	Where or evaluation may have introduced skew
Licence	What you're legally permitted to do with the model

The model card is your audit trail. In the absence of one, there is no audit trail, just a black box and the hope that whoever built it was thorough. In security, hope is not a control.

Answer the questions below

What documentation artifact accompanies a model to describe what it is, how it was built, and where it falls short?

Model Card

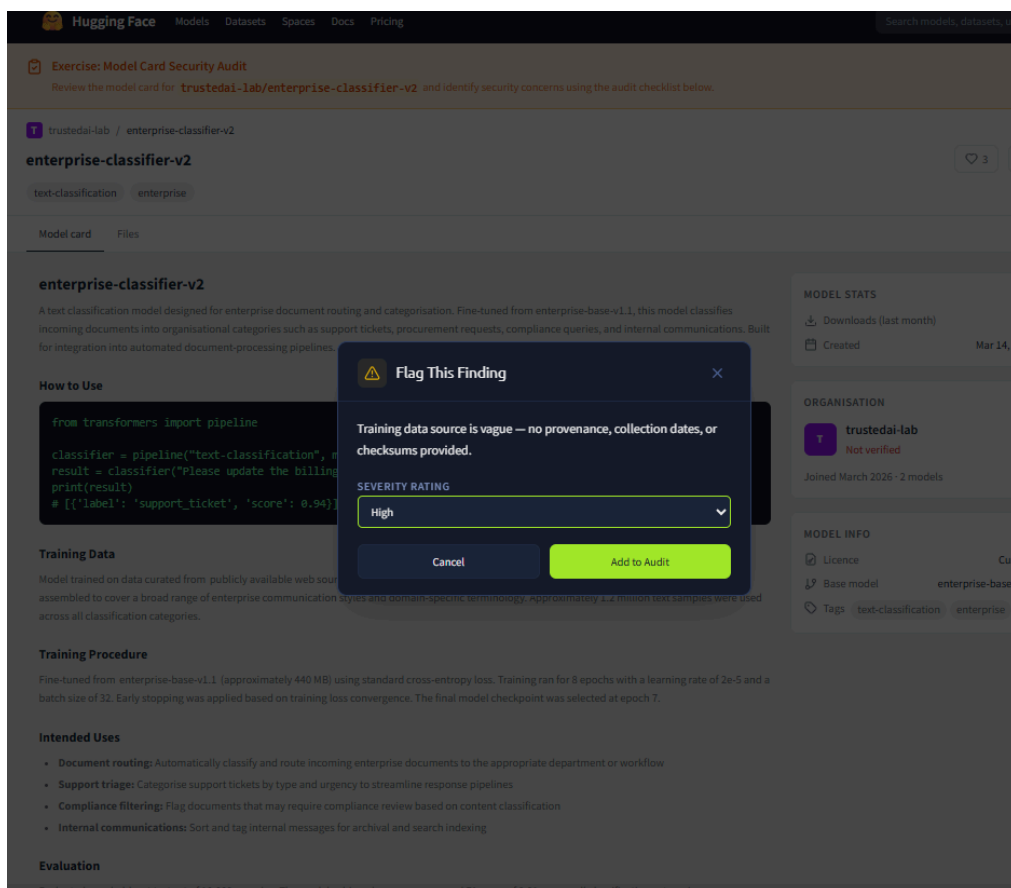
What are the billions of floating-point numbers that make up a trained model collectively referred to as?

Weights

Task 6 - Practical

Built to resemble **HuggingFace**, one of the most widely used platforms for hosting and sharing open-source AI models. Because anyone can upload a model, it's a powerful resource and a real supply chain risk.

Complete the exercise to get the flag! THM{A_m0del_Stud3nt}



Hugging Face

Models

Datasets

Spaces

Docs

Pricing

Search models, datasets, users...

Exercise: Model Card Security Audit

Review the model card for **trustedai-lab/enterprise-classifier-v2** and identify security concerns using the audit checklist below.

trustedai-lab

/

enterprise-classifier-v2

enterprise-classifier-v2

3

Follow

text-classification

enterprise

Model card

Files

enterprise-classifier-v2

A text classification model designed for enterprise document routing and categorisation. Fine-tuned from enterprise-base-v1.1, this model classifies incoming documents into organisational categories such as support tickets, procurement requests, compliance queries, and internal communications. Built for integration into automated document-processing pipelines.

How to Use

```
from transformers import pipeline

classifier = pipeline("text-classification",
result = classifier("Please update the billing
print(result)
# [{"label": "support_ticket", "score": 0.94}]
```

Training Data

Model trained on data curated from publicly available web sources assembled to cover a broad range of enterprise communication styles and domain-specific terminology. Approximately 4.6 million text samples were used across all classification categories.

Training Procedure

Fine-tuned from enterprise-base-v1.1 (approximately 440 MB) using standard cross-entropy loss. Training ran for 8 epochs with a learning rate of 2e-5 and a batch size of 32. Early stopping was applied based on training loss convergence. The final model checkpoint was selected at epoch 7.

Intended Uses

- Document routing: Automatically classify and route incoming enterprise documents to the appropriate department or workflow
- Support triage: Categorise support tickets by type and urgency to streamline response pipelines
- Compliance filtering: Flag documents that may require compliance review based on content classification
- Internal communications: Sort and tag internal messages for archival and search indexing

Evaluation

MODEL STATS

Downloads (last month)

127

Created

Mar 14, 2026

ORGANISATION

trustedai-lab

Not verified

Joined March 2026 · 2 models

MODEL INFO

Licence

Custom

Base model

enterprise-base-v1.1

Tags

text-classification

enterprise

Flag This Finding

No mention of PII filtering despite scraped forum and web data in the corpus.

SEVERITY RATING

Select severity...

Cancel

Add to Audit

Hugging Face

Models

Datasets

Spaces

Docs

Pricing

Search models, datasets, users...

Exercise: Model Card Security Audit

Review the model card for **trustedai-lab/enterprise-classifier-v2** and identify security concerns using the audit checklist below.

trustedai-lab

/

enterprise-classifier-v2

enterprise-classifier-v2

3

Follow

text-classification

enterprise

Model card

Files

enterprise-classifier-v2

A text classification model designed for enterprise document routing and categorisation. Fine-tuned from enterprise-base-v1.1, this model classifies incoming documents into organisational categories such as support tickets, procurement requests, compliance queries, and internal communications. Built for integration into automated document-processing pipelines.

How to Use

```
from transformers import pipeline

classifier = pipeline("text-classification",
result = classifier("Please update the billing
print(result)
# [{"label": "support_ticket", "score": 0.94}]
```

Training Data

Model trained on data curated from publicly available web sources assembled to cover a broad range of enterprise communication styles and domain-specific terminology. Approximately 4.6 million text samples were used across all classification categories.

Training Procedure

Fine-tuned from enterprise-base-v1.1 (approximately 440 MB) using standard cross-entropy loss. Training ran for 8 epochs with a learning rate of 2e-5 and a batch size of 32. Early stopping was applied based on training loss convergence. The final model checkpoint was selected at epoch 7.

Intended Uses

- Document routing: Automatically classify and route incoming enterprise documents to the appropriate department or workflow
- Support triage: Categorise support tickets by type and urgency to streamline response pipelines
- Compliance filtering: Flag documents that may require compliance review based on content classification
- Internal communications: Sort and tag internal messages for archival and search indexing

Evaluation

MODEL STATS

Downloads (last month)

127

Created

Mar 14, 2026

ORGANISATION

trustedai-lab

Not verified

Joined March 2026 · 2 models

MODEL INFO

Licence

Custom

Base model

enterprise-base-v1.1

Tags

text-classification

enterprise

Flag This Finding

Fine-tuned from enterprise-base-v1.1 — a specific version with no documentation of what that version contains or whether it has known issues.

SEVERITY RATING

Medium

Cancel

Add to Audit

AI Security 1.3. AI Models & Data

8

 trustedai-lab / enterprise-classifier-v2

enterprise-classifier-v2

3

Follow

text-classification

enterprise

Model card

Files

enterprise-classifier-v2

A text classification model designed for enterprise document routing and categorisation. Fine-tuned from enterprise-base-v1.1, this model classifies incoming documents into organisational categories such as support tickets, procurement requests, compliance queries, and internal communications. Built for integration into automated document-processing pipelines.

How to Use

```

from transformers import pipeline

classifier = pipeline("text-classification", model="enterprise-classifier-v2")
result = classifier("Please update the billing details for my account.")
print(result)
# [{"label": "support_ticket", "score": 0.94}]

```

Training Data

Model trained on data curated from publicly available web sources, assembled to cover a broad range of enterprise communication across all classification categories.

Training Procedure

Fine-tuned from enterprise-base-v1.1 (approximately 440 MB) using a batch size of 32. Early stopping was applied based on training loss convergence. The final model checkpoint was selected at epoch 7.

Intended Uses

- Document routing:** Automatically classify and route incoming enterprise documents to the appropriate department or workflow
- Support triage:** Categorise support tickets by type and urgency to streamline response pipelines
- Compliance filtering:** Flag documents that may require compliance review based on content classification
- Internal communications:** Sort and tag internal messages for archival and search indexing

Evaluation

Evaluated on a held-out test set of 10,000 samples. The model achieved a macro-averaged F1 score of 0.91 across all classification categories.

Licence

Custom licence. Contact trustedai-lab for terms.

MODEL STATS

Downloads (last month)

127

Created

Mar 14, 2026

ORGANISATION

 trustedai-lab

Not verified

Joined March 2026 · 2 models

MODEL INFO

Licence

Custom

Base model

enterprise-base-v1.1

Tags

text-classification

enterprise



Flag This Finding

×

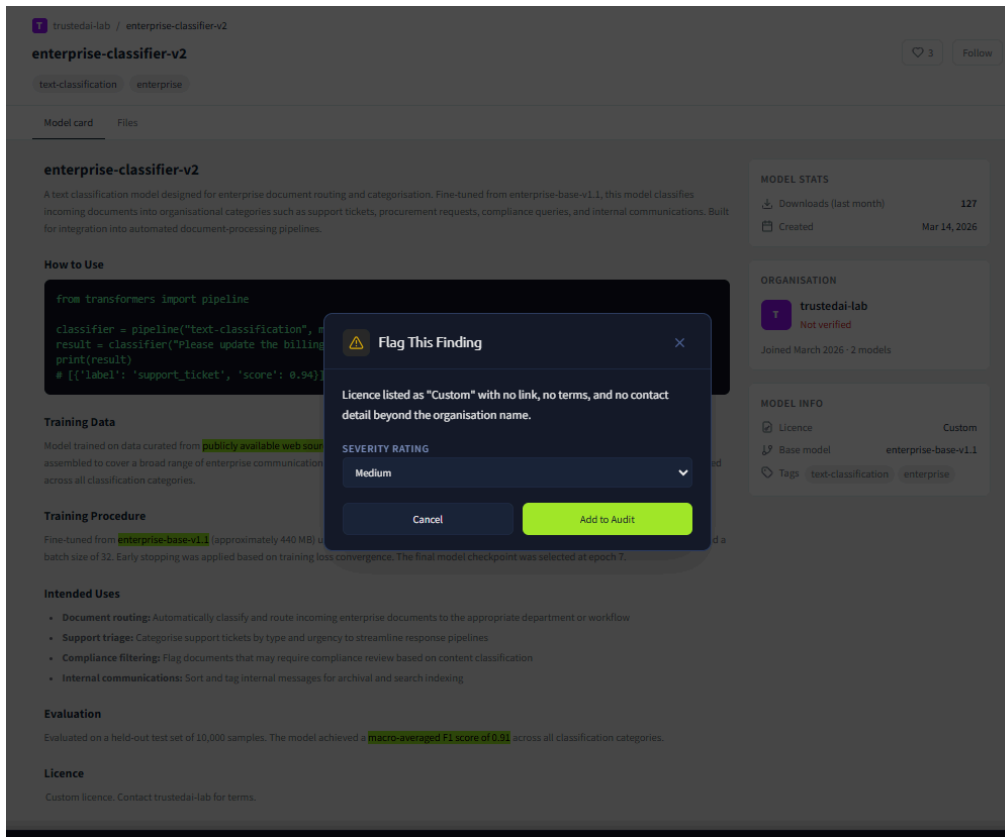
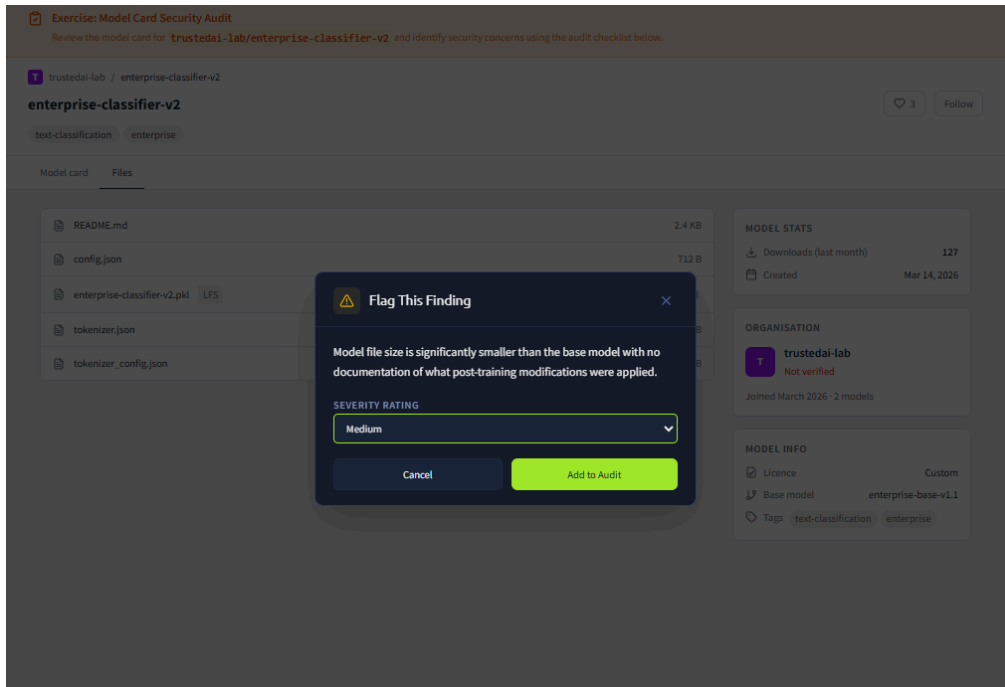
No bias evaluation, no limitations section, and a sparse evaluation section with no adversarial or red-team testing documented.

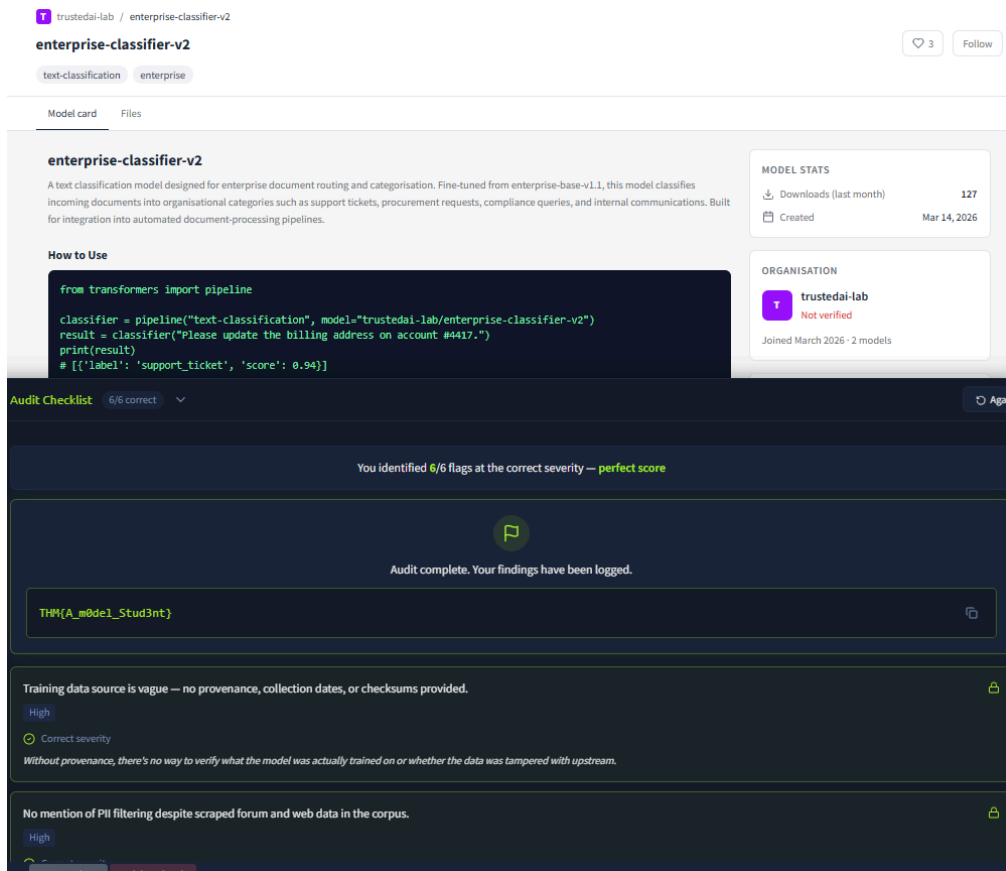
SEVERITY RATING

Medium

Cancel

Add to Audit





Task 7 - Conclusion

Explored how the security risks of AI systems are shaped long before a model ever reaches production. From the unaudited data pipelines that feed pre-training, to the inherited vulnerabilities carried through fine-tuning, to the simple fact that no one can look inside a trained model and see what it's actually doing, the risks stack up at every stage of the build process, long before anyone thinks to ask a security question. With this foundation, you're better equipped to ask the right questions when considering adoption or deployment of : where did this model come from, what does it contain, and does anyone actually know?

Here's a summary of this room's key takeaways:

- AI training data is drawn from poorly documented, unaudited sources, meaning most organisations have no reliable answer to where their training data came from, what it contained, or whether it was tampered with
- PII and live credentials routinely end up baked into model weights through large-scale web scraping and cannot be patched out once the model is

deployed

- Model-building decisions such as quantisation and federated learning introduce security trade-offs that are rarely documented, meaning organisations inherit unknown behaviour modifications alongside efficiency gains
- Fine-tuning a pre-trained model inherits everything beneath it: safety alignment erodes with as few as 10 adversarial examples, and fine-tuned models are measurably more susceptible to prompt injection than their base counterparts
- Trained model weights are fundamentally opaque; security testing can only sample behaviour rather than audit it, and model cards (the primary transparency mechanism) remain voluntary, frequently incomplete, and sometimes absent entirely